Introduction to Data Structures and Algorithms

Adopted from Clifford A. Shaffer. Data Structures and Algorithm Analysis. Chapter 1

Goals

- Primary goals:
  - Demonstrate a familiarity with major algorithms and data structures.
  - Apply important algorithmic design paradigms and methods of analysis (Tradeoffs between space and time)
  - Synthesize efficient algorithms in common engineering design situations. (Computational analysis)

Objectives

- To design an algorithm that is easy to understand, code, and debug. (No)

- To design an algorithm that makes efficient use of the computer's resources. (Yes)

Write efficient Programs

The Need for Data Structures

- Data structures: data representation and its associated operations.
- Data structures organize data

  $\Rightarrow$ more efficient programs.
- More powerful computers

  $\Rightarrow$ more complex applications.
- More complex applications

  $\Rightarrow$ more computation

Efficiency

- Choice of data structure or algorithm can make the difference between a program running in a few seconds or many days.
- A solution is said to be efficient if it solves the problem within its resource constraints.
  - Space
  - Time
- The cost of a solution is the amount of resources that the solution consumes.

Selecting a Data Structure

Select a data structure as follows:

1. Analyze the problem to determine the basic operations that must be supported.
2. Quantify the resource constraints for each operation.
3. Select the data structure that best meets these requirements.

Some Questions to Ask

- Are all data inserted into the data structure at the beginning, or are insertions interspersed with other operations?
- Can data be deleted?
- Are all data processed in some well-defined order, or is random access allowed?

Costs and Benefits

- Each data structure has costs and benefits.
- Rarely is one data structure better than another in all situations.
- Any data structure requires:
  - space for each data item it stores,
  - time to perform each basic operation,
  - programming effort.

Costs and Benefits (cont)

- Each problem has constraints on available space and time.
- Only after a careful analysis of problem characteristics can we know the best data structure for the task.
- Bank example:
  - Open account: a few minutes
  - Transactions: a few seconds
  - Close account: overnight

Abstract Data Types

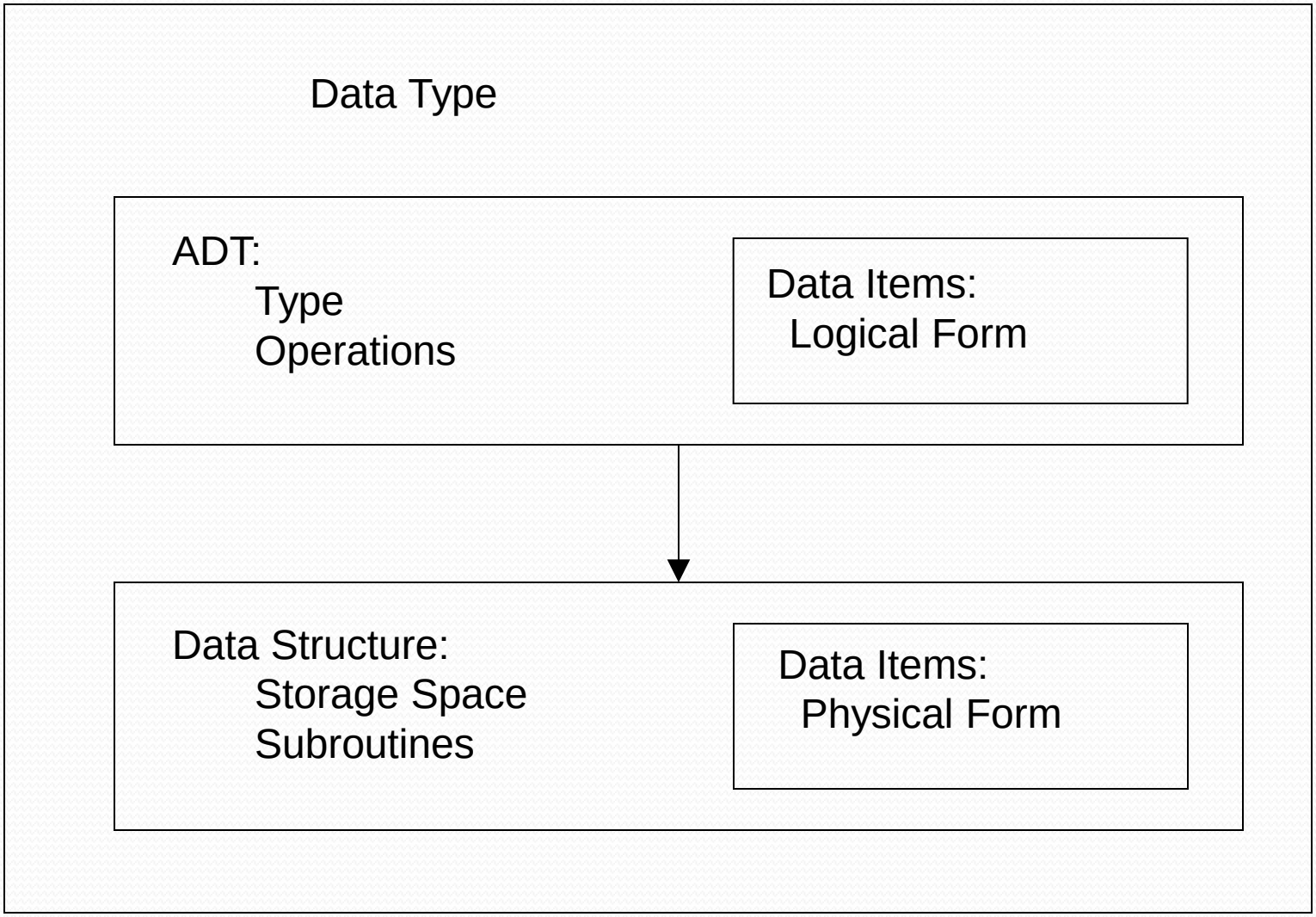
- Type: a collection of values.
- Data type: a type and a collection of operations to manipulate the type.
- Abstract Data Type (ADT): a definition for a data type solely in terms of a set of values and a set of operations on that data type.
- Each ADT operation is defined by its inputs and outputs.
- Encapsulation: Hide implementation details.

Data Structure

- Data structure is the physical implementation of an ADT.
  - Each operation associated with the ADT is implemented by one or more subroutines in the implementation.
- Data structure usually refers to an organization for data in main memory.
- File structure: an organization for data on peripheral storage, such as a disk drive.

Logical vs. Physical Form

- Data items have both a logical and a physical form.
- Logical form: definition of the data item within an ADT.
  - Ex: Integers in mathematical sense: +, -
- Physical form: implementation of the data item within a data structure.
  - Ex: 16/32 bit integers, overflow.


Data Type
ADT:
Type
Operations
Data Items:
Logical Form
Data Structure:
Storage Space
Subroutines
Data Items:
Physical Form

Problems, Algorithms, and Programs

- A problem is a task to be performed.
- A problem is a function or a mapping of inputs to outputs.
- An algorithm is a method or a process followed to solve a problem.
- Algorithms must be correct, of finite length, and must terminate for all inputs.

Problems

- Problem: a task to be performed.
  - Best thought of as inputs and matching outputs.
  - Problem definition should include constraints on the resources that may be consumed by any acceptable solution.
- Problems $\Leftrightarrow$ mathematical functions
  - A function is a matching between inputs (the domain) and outputs (the range).

Algorithms and Programs

- Algorithm: a method or a process followed to solve a problem.
  - A recipe.
- An algorithm takes the input to a problem (function) and transforms it to the output.
  - A mapping of input to output.
- A problem can have many algorithms.
- A computer program is an instance, representation, of an algorithm in some programming language.